竞赛编程速查表

程序 数值 容器 字符串

完整程序与读入输出

C++17。片段中 a 默认从 0 开始，n 为元素个数；按题意补输入、初值和合法性判断。

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <queue>
#include <functional>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n);
    for (int& x : a) cin >> x;
    ll answer = 0; // compute the real answer
    cout << answer << '\n';
    return 0;
}
```

answer 换成你算出的答案变量。每个输入一般只处理一组；只有题面给 T 时才循环 T 组。输出不加提示文字。

先估范围 再选类型

int 上限 2147483647；long long 上限约 9.22×10^18。20 万个 10^9 的总和可达 2×10^14。

```
ll sum = 0;
ll product = 1LL * x * y;
ll square = 1LL * x * x;
ll best = max(0LL, 1LL * a[0]);
const ll INF = (1LL << 60);
```

乘法必须先提升运算类型；long long 接收一个已经溢出的 int 结果，救不回来。max 的两个实参类型应一致。INF 要覆盖范围，别对无效哨兵直接做加法。

取模与完整大整数

```
ll MOD = 1000000007;
ll r = ((x % MOD) + MOD) % MOD;
ll add = (u % MOD + v % MOD) % MOD;
```

以上模数下两项相加安全；递推每一步取模。乘积仍须估范围，不能因最后取模就忽略中间溢出。需要完整超长整数时用 string 逐位运算，stoi/stoll 也有范围。

vector 的大小与下标

```
vector<int> a(n, 0);
vector<ll> pre(n + 1, 0);
vector<vector<int>> vis(
    H, vector<int>(W, 0));
a.push_back(x);
int last = a.back();
a.pop_back();
int n = (int)a.size();
```

长度为 n 的数组合法下标是 0 到 n-1。先读 H/W，再分配网格；变量变大不会让旧 vector 自动扩容。reserve 只预留空间，不能代替 resize。back/pop_back 前保证非空。

```
for (int& x : a) cin >> x;
for (const auto& x : a) {
    cout << x << ' ';
}
// 1-based storage
vector<int> b(n + 1);
for (int i = 1; i <= n; ++i)
    cin >> b[i];
```

用 1 起点就多开一格；用 0 起点就单独检查 n=1 时是否访问了下标 1。

pair 与结构化绑定

```
#include <utility>
pair<int, int> pt = {2, 7};
cout << pt.first << ' ' << pt.second;
auto [x, y] = pt;
vector<pair<int, ll>> events;
```

改过 pair 的结构后，逐处核对 first/second 的含义。

string 常用操作

```
string s, t;
cin >> s >> t;
int n = (int)s.size();
char ch = s[i];
int digit = s[i] - '0';
int ascii = (unsigned char)s[i];
string number = to_string(123);
string sub = s.substr(l, r - l + 1);
s += t;
reverse(s.begin(), s.end());
bool less = (s < t);
```

substr 第一个参数是 0 起点，第二个是长度。字符串比较按字典序，不按数值大小。字符数字转值用减 '0'；字母转 ASCII 数字串用 to_string(int(ch))。

```
getline(cin >> ws, s);
```

这一写法跳过前导空白再读整行。若前导空格本身有意义，应只清理前一行剩余换行后再 getline。

竞赛编程速查表 2

排序 二分 哈希 栈 队列 堆

排序与比较函数

```
sort(a.begin(), a.end());
sort(a.begin(), a.end(), greater<int>());
sort(v.begin(), v.end(),
     [](const auto& x, const auto& y) {
         return x.second < y.second;
     });
```

pair 默认先比 first，再比 second。比较函数用严格关系 < 或 >，不要写 <=。排序会改变原顺序，需要位置时一起保存原下标。

有序数组上的二分接口

```
auto lo = lower_bound(
    a.begin(), a.end(), x);
auto hi = upper_bound(
    a.begin(), a.end(), x);
int first = (int)(lo - a.begin());
int count = (int)(hi - lo);
bool found = lo != a.end() && *lo == x;
```

前提：a 按升序排列。lower_bound 找第一个 >=x；upper_bound 找第一个 >x。没找到位置可能是 n，end() 不能解引用。

哈希计数与首次位置

```
#include <unordered_map>
unordered_map<int, int> freq;
for (int x : a) ++freq[x];
auto it = freq.find(x);
int cnt = (it == freq.end()
           ? 0 : it->second);

unordered_map<int, int> firstPos;
for (int i = 0; i < n; ++i) {
    if (!firstPos.count(a[i]))
        firstPos[a[i]] = i;
}
```

键是查询值，值是次数/位置等信息。预处理只建一次；不能每次询问重扫数组。mp[x] 在键不存在时会插入默认值；只查询可用 find。哈希平均 O(1)，最坏不保证。

复杂度快速估算

顺序做两遍 n：O(n)，不是 O(n^2)。两层各跑 n：O(n^2)。n×3 的状态仍是线性规模。n=20 万时，通常要避免枚举全部二元配对。

堆与动态最值

```
priority_queue<int> big;
priority_queue<int, vector<int>,
               greater<int>> small;
priority_queue<ll> pq(
    a.begin(), a.end());
pq.push(x);
if (!pq.empty()) {
    ll value = pq.top();
    pq.pop();
}
```

默认大根堆，top 是最大值；greater 是小根堆，top 是最小值。top 查看，pop 删除且不返回值；两者都要求非空。堆内部没有整体排序。

查看 O(1)，push/pop 各 O(log n)。范围构造整体建堆 O(n)；逐个 push 最坏 O(n log n)。后续反复修改仍有对数成本。

保留最大的 K 项：小根堆；新项入堆后，若超过 K 项就弹最小项。堆顶是保留项中最小者，即第 K 大。最小的 K 项则对称使用大根堆。

修改堆顶：先 top 保存、pop 删除，计算新值后 push 回去。合并后数值会变大时，堆元素和中间量都要估范围。

栈与队列接口

```
#include <stack>
stack<int> st;
st.push(x);
if (!st.empty()) {
    int value = st.top();
    st.pop();
}

queue<pair<int, int>> q;
q.push({sx, sy});
if (!q.empty()) {
    auto [x, y] = q.front();
    q.pop();
}
```

栈后进先出，用 top；队列先进先出，用 front。queue.pop() 删除队首，不能拿 back() 当成被删除的元素。括号匹配遇右括号先检查空栈。

调试时先看最短输入

n=1、K=1、K=N、全相同、全负数、无解、答案为 0。结果错误先核对变量含义、更新顺序和初值，再检查多加/少加了一次。

竞赛编程速查表 3

前缀和 差分 事件 双指针 窗口

前缀和与区间查询

```
vector<ll> pre(n + 1, 0);
for (int i = 0; i < n; ++i)
    pre[i + 1] = pre[i] + a[i];
// query [l, r], 1-based inclusive
ll sum = pre[r] - pre[l - 1];
// query [l, r], 0-based inclusive
ll sum0 = pre[r + 1] - pre[l];
```

pre[i] 是前 i 个数的和，pre[0]=0。查询端点的起点体系必须和公式一致。建表一次 O(n)，每次询问 O(1)。累计范围常比单项大。

闭区间加法与恢复

```
vector<ll> diff(n + 2, 0);
// add v to [l,r], 1-based
diff[l] += v;
diff[r + 1] -= v;

ll added = 0;
for (int i = 1; i <= n; ++i) {
    added += diff[i];
    result[i - 1] = a[i - 1] + added;
}
```

先记录全部区间操作，再累计恢复。added 表示当前位置受到的总增量；若原数组全 0，它就是恢复后的值。闭区间撤销在 r+1，多留一格。若还要问覆盖量的区间总和，对恢复结果再建一次前缀和。

稀疏时间事件与分段计费

```
#include <map>
map<ll, ll> delta;
delta[l] += cost;
delta[r + 1] -= cost;

ll cur = 0, ans = 0;
ll last = delta.begin()->first;
for (auto [t, change] : delta) {
    ans += min(cur, cap) * (t - last);
    cur += change;
    last = t;
}
```

上例为至少一个服务、闭区间按日收费并设每日上限 cap。先用旧 cur 结算 [last,t)，再更新。时间长度是 t-last，不是事件个数。半开区间 [l,r) 在 r 撤销；同一时刻增量合并后再检查容量。

相向双指针

```
sort(a.begin(), a.end());
int l = 0, r = n - 1;
while (l < r) {
    ll sum = 1LL * a[l] + a[r];
    if (sum == target) {
        // record the pair
        break;
    }
    if (sum < target) ++l;
    else --r;
}
```

适用此例的是有序数组中找一对目标和。l<r 保证不复用同一个位置；需要原下标时，排序前保存它。

固定长度窗口最大和

```
ll sum = 0;
for (int i = 0; i < k; ++i)
    sum += a[i];
ll best = sum;
for (int r = k; r < n; ++r) {
    sum += a[r];
    sum -= a[r - k];
    best = max(best, sum);
}
```

前提 1<=K<=N。必须选满 K 个；全负数组时答案仍为负，不能用 0 初始化最大值。

非负数组的至多和计数

```
ll sum = 0, count = 0;
int l = 0;
for (int r = 0; r < n; ++r) {
    sum += a[r];
    while (l <= r && sum > limit)
        sum -= a[l++];
    count += r - l + 1;
}
```

前提：a[i]>=0 且 limit>=0。固定 r 后，合法起点为 l..r，所以加 r-l+1，不是 res++。含负数时该单调窗口不能直接套。最长长度类则更新最大 r-l+1。

窗口内频次与恰好计数

最长无重复段：加入右端使某值频次变成 2，就不断移出左端并扣回频次，直到不重复。左右端移动必须同步更新状态。

恰好 K 个奇数：F(K)-F(K-1)，F 表示至多的区间数；F(-1)=0。先数清一个右端点能对应多少起点，不把条件成立次数当区间个数。

竞赛编程速查表 4

基础动态规划 网格搜索 提交检查

先写清状态含义

先确定 dp 表示哪一类合法方案，再列最后一步的候选。数组维度由必须保留的信息决定；先算各维乘积，再估时间和空间。最终答案不一定是最后一个状态。

不相邻取数 可空选

dp[i] 表示前 i 个数的最大总和，dp[0]=0。取当前位置就接前 i-2 个数，否则沿用前 i-1 个数。

```
vector<ll> dp(n + 1, 0);
for (int i = 1; i <= n; ++i) {
    ll prev = (i >= 2 ? dp[i - 2] : 0);
    dp[i] = max(dp[i - 1],
                prev + a[i - 1]);
}
ll answer = dp[n];
```

最大连续子数组 非空

ending 必须包含当前位置。只能独自开新段，或接在上一位置结尾的段后；跳过负数会破坏连续性。best 记录所有结尾中的最大值。

```
ll ending = a[0], best = ending;
for (int i = 1; i < n; ++i) {
    ending = max(1LL * a[i],
                 ending + a[i]);
    best = max(best, ending);
}
```

相邻两天不能选同活动

每天 3 项活动，dp[c] 表示截至上一天且上一天选 c 的最大收益。当天选 c 时，只从另外两种旧结尾转移；先算完整 next，再覆盖旧 dp。

```
#include <array>
array<ll, 3> dp = {
    v[0][0], v[0][1], v[0][2]};
for (int i = 1; i < n; ++i) {
    array<ll, 3> next;
    for (int c = 0; c < 3; ++c)
        next[c] = v[i][c] + max(
            dp[(c + 1) % 3],
            dp[(c + 2) % 3]);
    dp = next;
}
ll answer = max({dp[0], dp[1], dp[2]});
```

分类候选何时能压缩

固定右端点后，若所有同类旧候选都只差一个固定加数，只需保留该类最大候选值。先明确哪些旧位置已经合法，再加入；必须保留历史最大，不能直接覆盖。不要因为看见颜色就机械开 n×k 状态。

BFS 无权图最短步数

grid[x][y] 中 x 是行，y 是列，# 是墙。读完 H/W 后分配 dist。起点可走，四个方向每步代价均为 1。

```
int dx[4] = {1,-1,0,0};
int dy[4] = {0,0,1,-1};
vector<vector<int>> dist(
    H, vector<int>(W, -1));
queue<pair<int,int>> q;
dist[sx][sy] = 0;
q.push({sx,sy});
while (!q.empty()) {
    auto [x,y] = q.front(); q.pop();
    for (int d = 0; d < 4; ++d) {
        int nx=x+dx[d], ny=y+dy[d];
        if (nx<0 || nx>=H ||
            ny<0 || ny>=W) continue;
        if (grid[nx][ny]=='#' ||
            dist[nx][ny]!=-1) continue;
        dist[nx][ny]=dist[x][y]+1;
        q.push({nx,ny});
    }
}
```

入队时就标记，避免重复入队。终点 dist=-1 表示不可达。BFS 的这份实现用队列循环，不是递归。

DFS 的调用与返回

一层负责当前格：先标记，再逐个递归访问合法邻格。子调用结束后，回到调用后的下一句，继续本层循环。连通块计数中 visited 通常不撤销；枚举路径的回溯才按状态需要撤销。

大网格可能递归过深。只求可达或连通块时，也可用队列/显式栈遍历；普通 DFS 第一次找到终点的路径不保证最短。

提交前最后检查

① main 有读入和输出，函数确实被调用；所有非 void 路径有 return。② 单元素、全负、全相同、无解和最大数值。③ 下标 0/1 起点及 r+1 的容量。④ 队列 front 与 pop 对应。⑤ 累计量和中间乘积的类型。

赛前与赛场

关闭 IDE 自动补全、整行建议和模板展开。纸质资料可查；不要把开卷理解成可搜索网页。先扫题，优先做能明确写出步骤的题；卡住先留草稿，再换题。

规则来源：组委会 2026-11-10 指南 50362；赛事主页 11-06 回复允许纸质资料。

m.saikr.com/contest/notice_detail/50362

m.saikr.com/adpc2026autumn